

GITHUB link - [Narasimha3-saladi/Logbook_finance](#)

Week-1 Logbook:

Lab Logbook Requirement:

1) Create a vector using `np.arange`.

Determine the number of the vector elements using the following method: Take the last two digits from your SID. It should be from 00 to 99. If this number is 10 or more, it becomes the required number of the vector elements. If it is less than 10, add 100 to your number.

For example, if your SID is 2287467, and the last two digits are 67, which is greater than 10. The required number is 67. If your SID is 2287407, and the last two digits are 07, which is less than 10. The required number is 107.

Then,

2. Change matrix `a` to 2-d array with 1 row. Print the array. You should have the two sets of brackets for a 2-d array with one row.
3. Save it in another array. Print the array.
4. Check the shape attribute value.
5. Add the code and result to your Lab Logbook

¶

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB OR ONEDRIVE.

Logbook solution:

```
[67]: a = np.arange(14) #sid = 2367814 as the last two digits of the sid is greater than 10
      a1 = a.reshape(1, -1) #changing matrix a to 2-d array with 1 row
      print(a1)
      print('\n')
      print(a1.shape)
```

```
[[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13]]
```

```
(1, 14)
```

Week-2 Logbook:

Lab Logbook Requirement:

1. Determine a number (`n`) equal to the last digit of your SID.
2. Group by "relationship" and "hours-per-week".
3. Reduce all "hours-per-week" column values in the original DataFrame by the value '`n`'.
4. Group by "relationship" and reduced "hours-per-week".
5. Add the code and result to your Lab Logbook.

¶

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB OR ONEDRIVE.

Logbook solution:

```
[53]: #Group by before reducing hours
Group_by_relationship = data.groupby(["relationship", "hours-per-week"])
Group_by_relationship.size()
```

```
[53]: relationship  hours-per-week
Husband          13.0             1
          40.0             2
          45.0             1
          80.0             1
Not-in-family    16.0             1
          40.0             2
          50.0             2
Own-child        30.0             1
Wife             40.0             2
dtype: int64
```

```
[54]: n = 4 #sid = 2367814

def func(x):
    return x - n

data['hours-per-week'] = data['hours-per-week'].apply(func)
```

```
[55]: #Group by after reducing hours
Group_by_relationship = data.groupby(["relationship", "hours-per-week"])
Group_by_relationship.size()
```

```
[55]: relationship  hours-per-week
Husband          9.0             1
          36.0             2
          41.0             1
          76.0             1
Not-in-family    12.0             1
          36.0             2
          46.0             2
Own-child        26.0             1
Wife             36.0             2
dtype: int64
```

Week-3 Logbook:

Lab Logbook Requirement:

1) Draw a bicolour features interaction diagram between the columns with the numbers of the last and second to last digits of your SID, where:

#	Column
---	-----
1	Account length
2	Area code
3	International plan
4	Voice mail plan
5	Number vmail messages
6	Total day minutes
7	Total day calls
8	Total day charge
9	Total eve minutes
0	Total eve calls

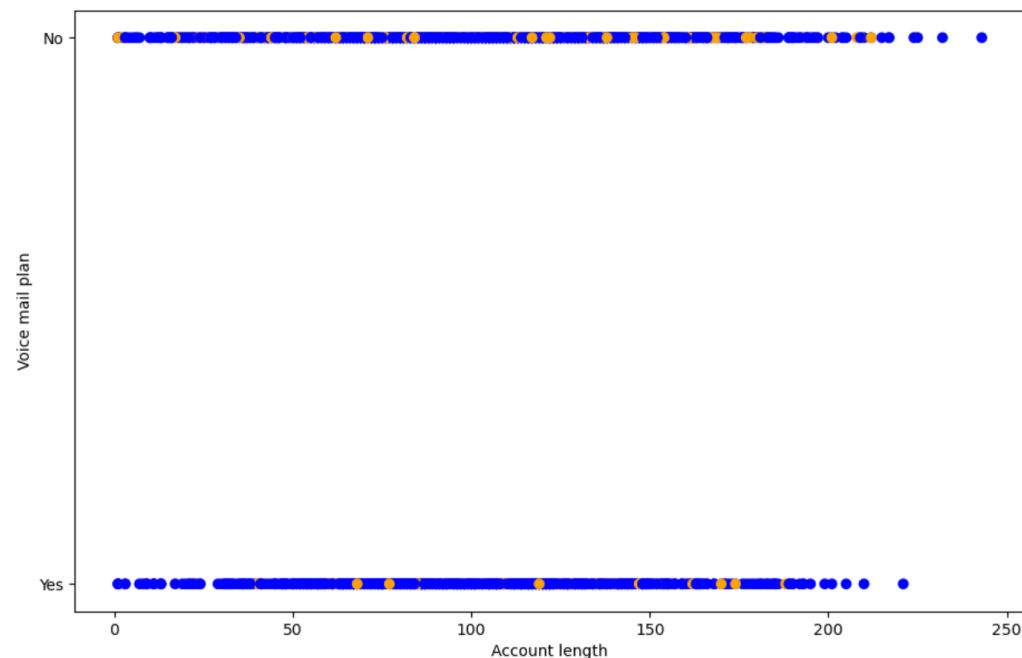
In case these numbers are the same, then take the next number in order as another column number. For example, if your SID is 2287477, then you plot the bicolour diagram of the 7th and 8th columns. If your SID is 2287499, then the 9th and 0.

2. Add the code and result to your Lab Logbook.

Logbook solution:

```
#sid = 2367814 so plotted the bicolour diagram of the 1st and 4th columns.
```

```
fig = plt.figure(figsize=(11,7))
plt.scatter(data['Account length'], data['Voice mail plan'], color = C1r);
plt.xlabel('Account length');
plt.ylabel('Voice mail plan');
```



Week-4 Logbook:

Lab Logbook Requirement:

1. Create your own Multi-layer Perceptron (MLP) with two hidden layers, where the first hidden layer cells' number equals the last three digits of your SID. The number of cells in the next hidden layer is approximately two times smaller. For example, if your SID is 2287167, the number of cells on the first hidden layer is 167, and on the second - 84. Take epochs=10. Leave other parameters the same as in the practical session.
2. Compile the model.
3. Train your MLP with the same datasets and demonstrate the received MAE.
4. Compare your MAE with the MAE of the MLP in the practical session.
5. Please only add to your Lab Logbook a print-screen of your MLP architecture using `model.summary()` and the resulting MAE.

¶

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB OR ONEDRIVE.

Logbook solution:

```
#sid - 2367814
```

```
model = keras.Sequential([
    keras.layers.Dense(814, input_dim = 500, activation = tf.nn.relu, kernel_initializer = "normal"),
    keras.layers.Dense(407, activation = 'relu', kernel_initializer = "normal"),
    keras.layers.Dense(1)
])

print(model.summary())
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 814)	407,814
dense_6 (Dense)	(None, 407)	331,705
dense_7 (Dense)	(None, 1)	408

Total params: 739,927 (2.82 MB)

Trainable params: 739,927 (2.82 MB)

Non-trainable params: 0 (0.00 B)

None

```

model.compile(optimizer = "adam", loss = "mse", metrics = ["mae"])

history = model.fit(X_train, y_train, batch_size = 10, epochs = 10, validation_split = 0.2, verbose = 1)

Epoch 1/10
2640/2640 ————— 22s 7ms/step - loss: 0.1024 - mae: 0.0653 - val_loss: 0.0172 - val_mae: 0.1174
Epoch 2/10
2640/2640 ————— 20s 7ms/step - loss: 2.8401e-04 - mae: 0.0129 - val_loss: 0.0180 - val_mae: 0.1204
Epoch 3/10
2640/2640 ————— 20s 8ms/step - loss: 3.1245e-04 - mae: 0.0134 - val_loss: 0.0131 - val_mae: 0.1054
Epoch 4/10
2640/2640 ————— 20s 7ms/step - loss: 2.1118e-04 - mae: 0.0110 - val_loss: 0.0059 - val_mae: 0.0688
Epoch 5/10
2640/2640 ————— 19s 7ms/step - loss: 1.0671e-04 - mae: 0.0079 - val_loss: 0.0013 - val_mae: 0.0286
Epoch 6/10
2640/2640 ————— 22s 8ms/step - loss: 8.2932e-05 - mae: 0.0070 - val_loss: 0.0022 - val_mae: 0.0424
Epoch 7/10
2640/2640 ————— 24s 9ms/step - loss: 7.3572e-05 - mae: 0.0065 - val_loss: 0.0014 - val_mae: 0.0335
Epoch 8/10
2640/2640 ————— 37s 8ms/step - loss: 6.0344e-05 - mae: 0.0059 - val_loss: 8.7449e-04 - val_mae: 0.0253
Epoch 9/10
2640/2640 ————— 21s 8ms/step - loss: 5.5353e-05 - mae: 0.0056 - val_loss: 9.6368e-04 - val_mae: 0.0274
Epoch 10/10
2640/2640 ————— 18s 7ms/step - loss: 5.9107e-05 - mae: 0.0058 - val_loss: 6.2315e-04 - val_mae: 0.0214

print("Mean absolute error: %.5f" % mae)

Mean absolute error: 0.05109

```

MAE of the MLP in the practical session:

```

mse, mae = model.evaluate(X_test, y_test, verbose = 0)

print("Mean absolute error: %.5f" % mae)
print("Mean squared error: %.5f" % mse)

```

```

Mean absolute error: 0.05109
Mean squared error: 0.00311

```

Week-5 Logbook:

Lab Logbook Requirement:

1. Modify the practical session CNN model by reducing the convolutional core size to 5.
2. Change the batch_size to 50.
3. Also, change the size of the number of epochs, which is calculated by the formula:
$$Z + Y, \text{ if } Z = 0$$
$$10 + Y, \text{ if } Z = 0 \text{ and } Y \text{ is not } 0$$
$$10, \text{ if } Z = Y = 0$$

, where your SID is: XXXXXZY
4. Leave other parameters the same as in the practical session.
5. Compile the model.
6. Train your CNN with the same datasets and demonstrate the received test MAE. Compare your MAE with the MAE of the CNN in the practical session.
7. Please only add a print-screen of your CNN architecture using model.summary() and the resulting MAE to your Lab Logbook.

Logbook solution:

```
model = keras.Sequential([
    keras.layers.Conv1D(50,5, padding = 'same', input_shape= (50,5), activation=tf.nn.relu, kernel_initializer='normal'),
    keras.layers.MaxPooling1D(7),
    keras.layers.Conv1D(100,5,padding = 'same', activation = tf.nn.relu, kernel_initializer = "normal"),
    keras.layers.GlobalMaxPooling1D(),
    keras.layers.Dense(25, activation =tf.nn.relu, kernel_initializer = "normal"),
    keras.layers.Dense(2)
])
print(model.summary())
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv1d_2 (Conv1D)	(None, 50, 50)	1,300
max_pooling1d_1 (MaxPooling1D)	(None, 7, 50)	0
conv1d_3 (Conv1D)	(None, 7, 100)	25,100
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 100)	0
dense_2 (Dense)	(None, 25)	2,525
dense_3 (Dense)	(None, 2)	52

Total params: 28,977 (113.19 KB)

Trainable params: 28,977 (113.19 KB)

Non-trainable params: 0 (0.00 B)

None

```
model.compile(optimizer = "adam", loss = "mse", metrics = ["mae"])
```

```
history = model.fit(X_train, y_train, batch_size =50, epochs=5, validation_split=0.2, verbose=1)
```

```
#sid = 2367814 where Z = 1 and Y = 4
```

```
Epoch 1/5  
3520/3520 ————— 33s 8ms/step - loss: 0.0086 - mae: 0.0446 - val_loss: 9.4102e-04 - val_mae: 0.0201  
Epoch 2/5  
3520/3520 ————— 32s 9ms/step - loss: 7.6485e-04 - mae: 0.0191 - val_loss: 9.0723e-04 - val_mae: 0.0202  
Epoch 3/5  
3520/3520 ————— 38s 11ms/step - loss: 7.0936e-04 - mae: 0.0180 - val_loss: 8.9222e-04 - val_mae: 0.0198  
Epoch 4/5  
3520/3520 ————— 27s 8ms/step - loss: 7.0750e-04 - mae: 0.0178 - val_loss: 0.0011 - val_mae: 0.0233  
Epoch 5/5  
3520/3520 ————— 26s 8ms/step - loss: 7.0204e-04 - mae: 0.0179 - val_loss: 8.2372e-04 - val_mae: 0.0184
```

Logbook MAE:

```
mse,mae = model.evaluate(X_test, y_test, verbose =1)  
print("Mean absolute error: %.5f" %mae)
```

```
936/936 ————— 3s 3ms/step - loss: 0.0012 - mae: 0.0229  
Mean absolute error: 0.02443
```

Practical session MAE:

```
mse,mae = model.evaluate(X_test, y_test, verbose =1)  
print("Mean absolute error: %.5f" %mae)
```

```
936/936 ————— 3s 3ms/step - loss: 0.0012 - mae: 0.0231  
Mean absolute error: 0.02464
```

Week-6 Logbook:

Lab Logbook Requirement:

1. Plot the price chart of the part of the whole dataset 'High_Bid' and 'Low_Bid' prices using `ipplot()` library.
2. The start point should equal the 5 last digits of your SID Number.
3. The time period (in minutes) should equal the 3 last digits of your SID Number.
4. Please only add a print-screen of your code and final graph to your Lab Logbook.

¶

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB OR ONEDRIVE.

Logbook solution:

```
#sid = 2367814

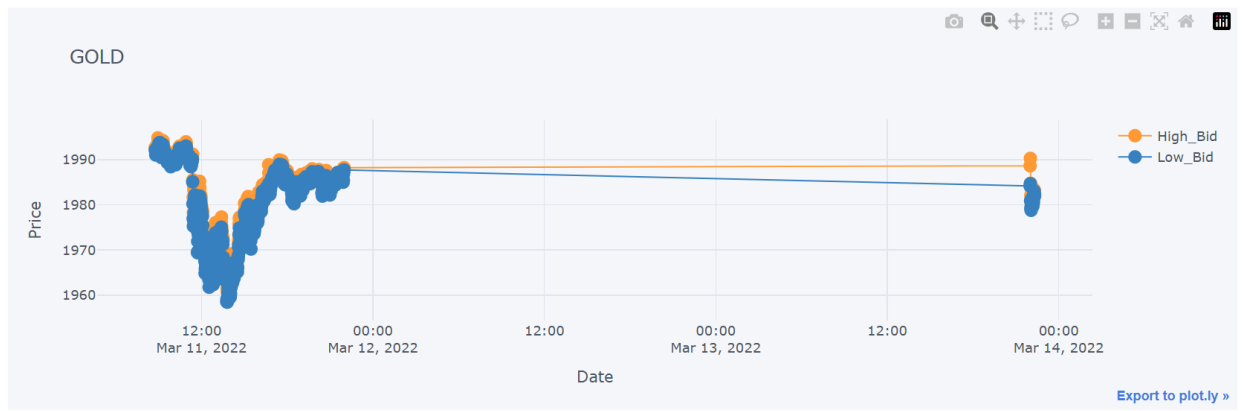
#Last 5 digits = 67814

#Last 3 digits = 814

data = data.iloc[67814:67814+814, :].copy()

data['Local_time_T'] = data['Local_time_T'].astype(str)

data[['High_Bid', 'Low_Bid', 'Local_time_T']].iplot(
    x='Local_time_T',
    y=['High_Bid', 'Low_Bid'],
    mode='lines+markers',
    xTitle='Date',
    yTitle='Price',
    title='GOLD'
)
```



Week-7 Logbook:

Lab Logbook Requirement:

1. Modify the practical session LSTM model parameter from 100 to be calculated using the formula:
 $ZY + 10$, where your SID is: XXXXXZY
2. Change the epochs to 10.
3. Change the patience to 3
4. Leave other parameters the same as in the practical session.
5. Compile the model.
6. Train your LSTM with the same datasets and demonstrate the received test MSE & MAE. Compare your test MSE & MAE with the MSE & MAE of the LSTM in the practical session.
7. Please only add to your Lab Logbook print-screens of:
 - your LSTM architecture using `model.summary()`,
 - the resulting test MSE & MAE and
 - MAE detailed graph

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB OR ONEDRIVE.

Logbook solution:

```
#sid =2367814 where Z=1 and Y=4  
#ZY +10 = 14 + 10 = 24
```

```
model = keras.Sequential([  
    keras.layers.LSTM(24, activation = 'relu', input_shape = (50, 18)),  
    keras.layers.Dense(2)  
)  
  
print(model.summary())
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
lstm_1 (LSTM)	(None, 24)	4,128
dense_1 (Dense)	(None, 2)	50

Total params: 4,178 (16.32 KB)

Trainable params: 4,178 (16.32 KB)

Non-trainable params: 0 (0.00 B)

None

```
model.compile(optimizer = "adam", loss = "mse", metrics = ["mae"])
```

```
es = EarlyStopping(monitor='val_loss', mode='min', patience=3, verbose=1)  
mc = ModelCheckpoint('best_model_LSTM_GOLD.keras', monitor='val_loss', mode='min', verbose=1, save_best_only=True)
```

```
history = model.fit(X_train, y_train, batch_size = 20, epochs = 10, validation_split = 0.1, shuffle = True, verbose = 1, callbacks = [es,mc])
```

```
Epoch 1/10  
1211/1213 ----- 0s 18ms/step - loss: 1.8554 - mae: 0.7152  
Epoch 1: val_loss improved from inf to 0.00016, saving model to best_model_LSTM_GOLD.keras  
1213/1213 ----- 27s 19ms/step - loss: 1.8520 - mae: 0.7140 - val_loss: 1.5804e-04 - val_mae: 0.0098  
Epoch 2/10  
1213/1213 ----- 0s 15ms/step - loss: 1.3798e-04 - mae: 0.0092  
Epoch 2: val_loss improved from 0.00016 to 0.00008, saving model to best_model_LSTM_GOLD.keras  
1213/1213 ----- 19s 16ms/step - loss: 1.3796e-04 - mae: 0.0092 - val_loss: 7.9576e-05 - val_mae: 0.0073  
Epoch 3/10  
1213/1213 ----- 0s 14ms/step - loss: 6.1001e-05 - mae: 0.0062  
Epoch 3: val_loss improved from 0.00008 to 0.00003, saving model to best_model_LSTM_GOLD.keras  
1213/1213 ----- 17s 14ms/step - loss: 6.0994e-05 - mae: 0.0062 - val_loss: 3.2284e-05 - val_mae: 0.0045  
Epoch 4/10  
1211/1213 ----- 0s 15ms/step - loss: 3.9329e-05 - mae: 0.0049  
Epoch 4: val_loss did not improve from 0.00003  
1213/1213 ----- 19s 16ms/step - loss: 3.9323e-05 - mae: 0.0049 - val_loss: 3.5733e-05 - val_mae: 0.0049  
Epoch 5/10  
1213/1213 ----- 0s 18ms/step - loss: 2.8951e-05 - mae: 0.0042  
Epoch 5: val_loss did not improve from 0.00003  
1213/1213 ----- 23s 19ms/step - loss: 2.8952e-05 - mae: 0.0042 - val_loss: 3.7234e-05 - val_mae: 0.0048  
Epoch 6/10  
1212/1213 ----- 0s 16ms/step - loss: 3.2684e-05 - mae: 0.0045  
Epoch 6: val_loss improved from 0.00003 to 0.00002, saving model to best_model_LSTM_GOLD.keras  
1213/1213 ----- 20s 17ms/step - loss: 3.2686e-05 - mae: 0.0045 - val_loss: 2.1792e-05 - val_mae: 0.0035  
Epoch 7/10  
1209/1213 ----- 0s 16ms/step - loss: 3.1002e-05 - mae: 0.0044  
Epoch 7: val_loss did not improve from 0.00002  
1213/1213 ----- 21s 17ms/step - loss: 3.1007e-05 - mae: 0.0044 - val_loss: 2.9551e-05 - val_mae: 0.0048  
Epoch 8/10  
1212/1213 ----- 0s 16ms/step - loss: 3.2873e-05 - mae: 0.0045  
Epoch 8: val_loss did not improve from 0.00002  
1213/1213 ----- 20s 16ms/step - loss: 3.2873e-05 - mae: 0.0045 - val_loss: 2.2040e-05 - val_mae: 0.0041  
Epoch 9/10  
1211/1213 ----- 0s 19ms/step - loss: 2.5428e-05 - mae: 0.0040  
Epoch 9: val_loss did not improve from 0.00002  
1213/1213 ----- 24s 19ms/step - loss: 2.5433e-05 - mae: 0.0040 - val_loss: 1.1025e-04 - val_mae: 0.0082  
Epoch 9: early stopping
```

```
scores = LSTM_saved_best_model.evaluate(X_test, y_test, verbose=1)
94/94 ————— 1s 13ms/step - loss: 8.6348e-07 - mae: 6.3720e-04
```

```
scores
```

```
[6.950400575078675e-07, 0.0005934922373853624]
```

```
print("Mean squared error (mse): %.9f " % (scores[0]))
```

```
Mean squared error (mse): 0.000000695
```

```
print("Mean absolute error (mae): %.9f " % (scores[1]))
```

```
Mean absolute error (mae): 0.000593492
```

```
history_dict = history.history
```

```
mae_values = history_dict['mae']
```

```
val_mae_values = history_dict['val_mae']
```

```
epochs = range(1, len(mae_values) + 1)
```

```
plt.figure(num=1, figsize=(15,7))
```

```
plt.plot(epochs, mae_values, 'b', label='Training Mean Absolute Error(MAE)')
```

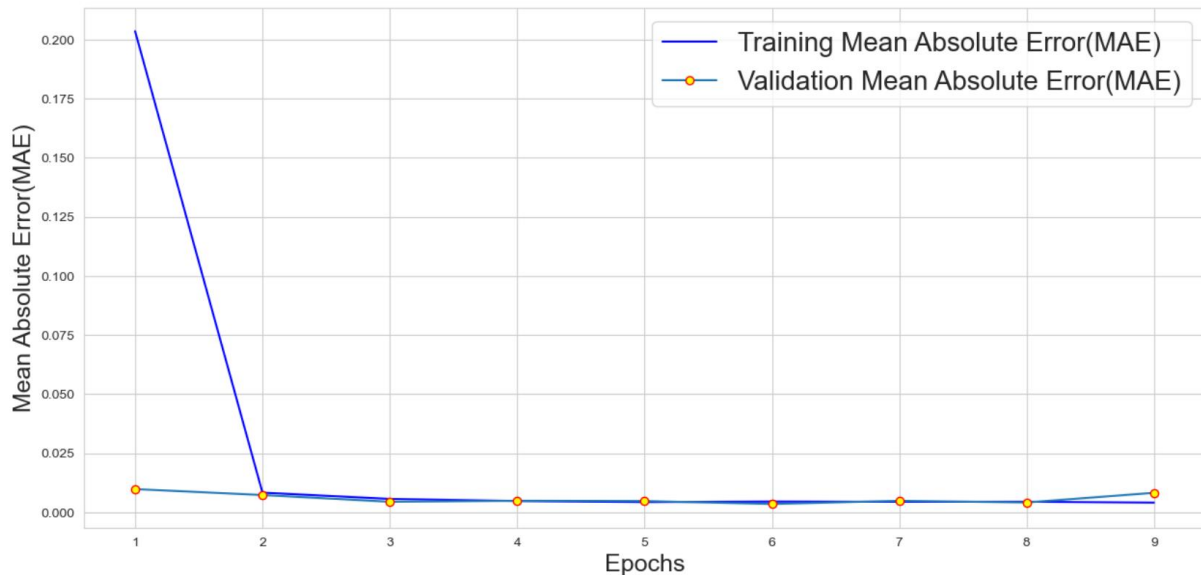
```
plt.plot(epochs, val_mae_values, marker='o', markeredgecolor='red', markerfacecolor='yellow', label='Validation Mean Absolute Error(MAE)')
```

```
plt.xlabel('Epochs', size=18)
```

```
plt.ylabel('Mean Absolute Error(MAE)', size=18)
```

```
plt.legend()
```

```
plt.show()
```

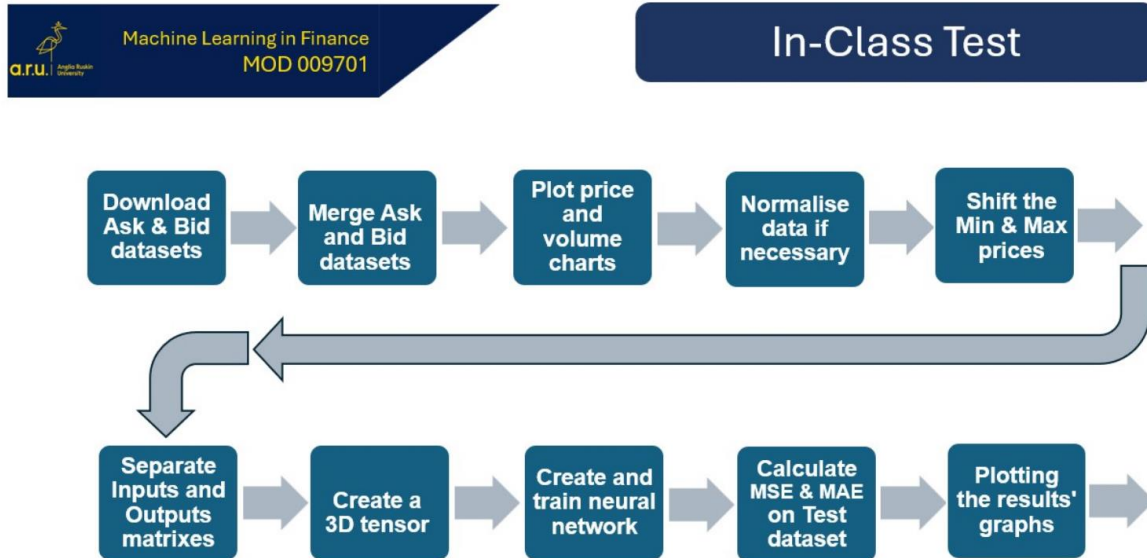


Week-8 Logbook:

Using the Mock Test Pipeline shown below, create and train a neural network of your choice (MLP, CNN, LSTM) and analyse and predict silver prices for the next 5-minute time-frame.

[XAGUSD 5 Mins Ask 2023.01.01 2023.06.30.csv](#) ↓

[XAGUSD 5 Mins Bid 2023.01.01 2023.06.30.csv](#) ↓



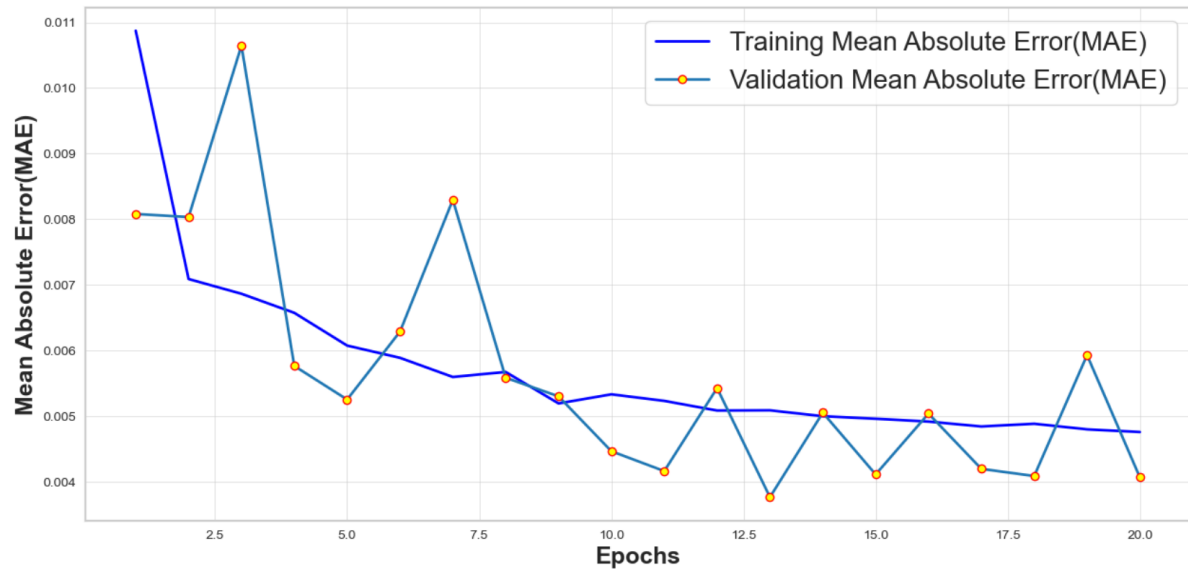
Lab Logbook requirements:

Please make sure to document the following in your Lablogbook:

Please only add to your Lab Logbook a print screen of the MAE resulting graph during the training process.

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB.

Logbook solution:



Week -10 Logbook:

Lab Logbook Requirement:

- *Plot 4 graphs:*
 1. *Precision during training graph*
 2. *More detailed Precision graph*
 3. *Training accuracy graph*
 4. *More detailed Accuracy graph*

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB OR ONEDRIVE.

Logbook solution:

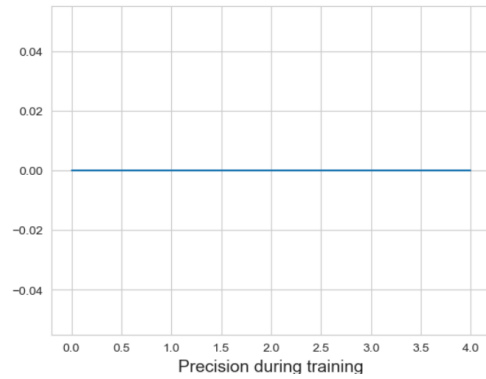
1. Precision during training graph:

Precision during training

[155]:

```
# Your code of Precision_score
#

plt.plot(history.history['precision'])
plt.xlabel('Precision during training', size = 14)
plt.show()
```

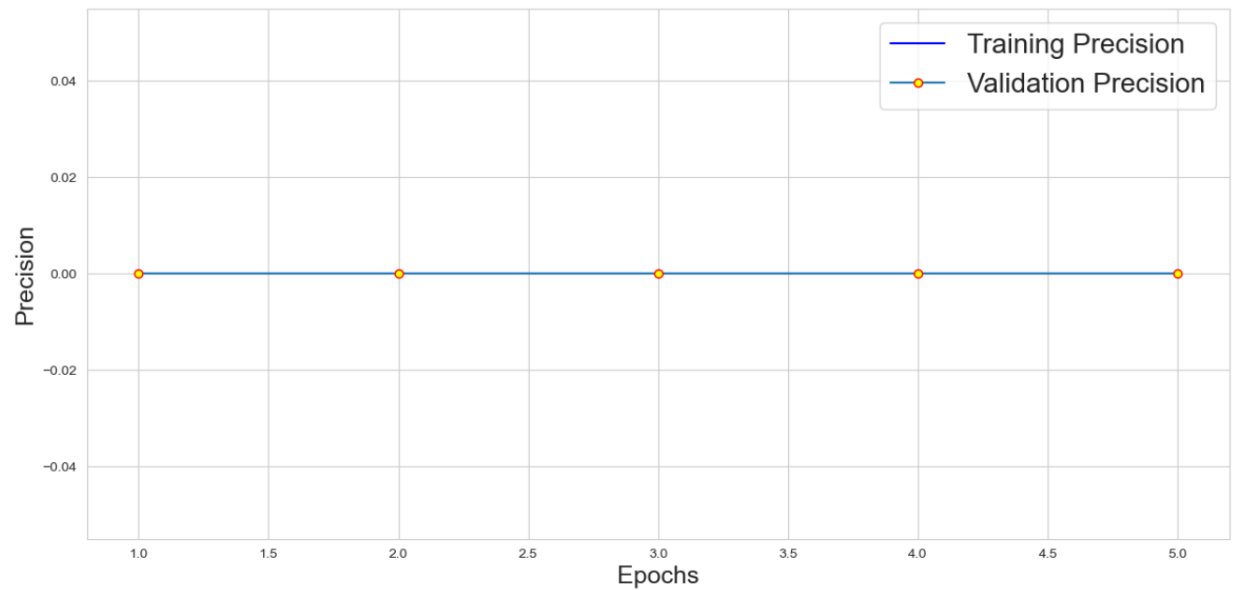


2. More detailed Precision graph

```
#
#
#   Your code of More detailed Precision
#
#
history_dict = history.history

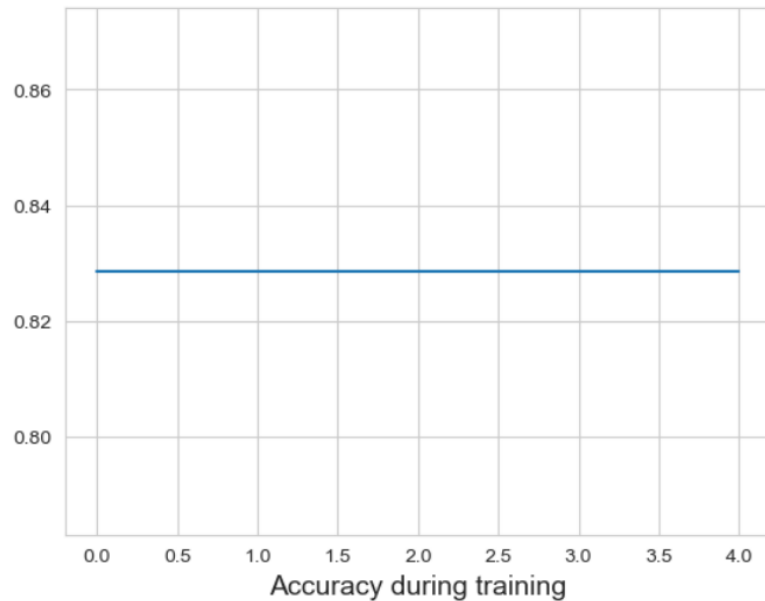
precision = history_dict['precision']
val_precision = history_dict['val_precision']

epochs = range(1, len(precision) + 1)
plt.figure(num=1, figsize=(15, 7))
plt.plot(epochs, precision, 'b', label='Training Precision', linestyle = '-')
plt.plot(epochs, val_precision, marker='o', markeredgcolor='red', markerfacecolor='yellow', label='Validation Precision')
plt.xlabel('Epochs', size=18)
plt.ylabel('Precision', size=18)
plt.legend()
plt.show()
```



3. Training accuracy graph

```
plt.plot(history.history['accuracy'])
plt.xlabel('Accuracy during training', size = 14)
plt.show()
```

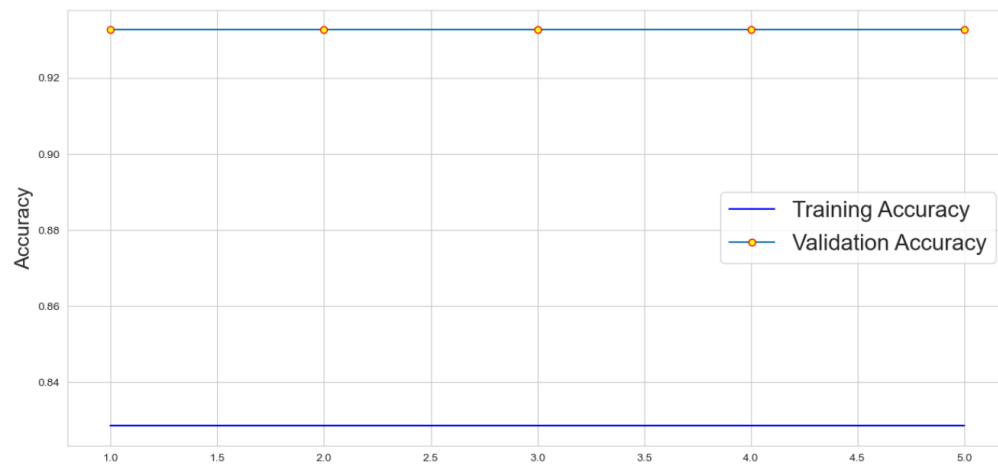


4. More detailed Accuracy graph

```
history_dict = history.history

accuracy = history_dict['accuracy']
val_accuracy = history_dict['val_accuracy']

epochs = range(1, len(history.history['accuracy']) + 1)
plt.figure(num=1, figsize=(15, 7))
plt.plot(epochs, accuracy, 'b', label='Training Accuracy')
plt.plot(epochs, val_accuracy, markers='o', markeredgecolor='red', markerfacecolor='yellow', label='Validation Accuracy')
plt.xlabel('Epochs', size=18)
plt.ylabel('Accuracy', size=18)
plt.legend()
plt.show()
```



Week-11 Logbook:

Lab Logbook requirements:

Please make sure to document the following in your Lablogbook:

1. Create and train your own LSTM model
2. Add all the LSTM's Error metrics: Accuracy, Precision, Recall, F1-Score and AUC to the final histogram "ML Models performance..."

NOTE: DON'T FORGET TO SAVE AND BACK UP YOUR COMPLETED JUPYTER NOTEBOOK AND LAB LOGBOOK ON GITHUB.

Logbook solution:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
import matplotlib.pyplot as plt

cr_data = pd.read_csv("credit_risk_dataset.csv")

cr_data['person_home_ownership'] = LabelEncoder().fit_transform(cr_data['person_home_ownership'])
cr_data['loan_intent'] = LabelEncoder().fit_transform(cr_data['loan_intent'])
cr_data['cb_person_default_on_file'] = LabelEncoder().fit_transform(cr_data['cb_person_default_on_file'])

cr_data.fillna(cr_data.median(), inplace=True)

X = cr_data.drop('loan_status', axis=1).values
y = cr_data['loan_status'].values

scaler = StandardScaler()
X = scaler.fit_transform(X)

X = X.reshape((X.shape[0], 1, X.shape[1]))

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = Sequential([
    LSTM(50, input_shape=(X_train.shape[1], X_train.shape[2]), activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=10, batch_size=32, verbose=1, validation_data=(X_test, y_test))

y_pred = (model.predict(X_test) > 0.5).astype(int)
```



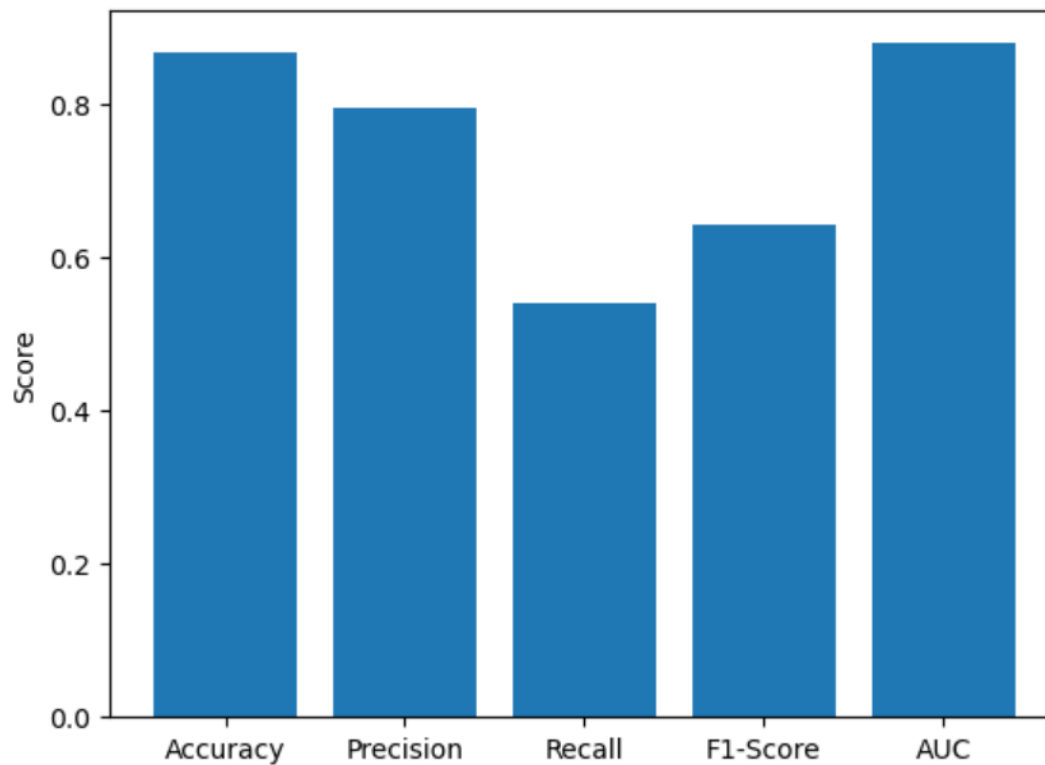
```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, model.predict(X_test))
```

```
metrics = {
    'Accuracy': accuracy,
    'Precision': precision,
    'Recall': recall,
    'F1-Score': f1,
    'AUC': auc
}
```

```
plt.bar(metrics.keys(), metrics.values())
plt.title("LSTM Model Performance")
plt.ylabel("Score")
plt.show()
```

Epoch 1/10
 815/815 ————— 11s 5ms/step - accuracy: 0.8194 - loss: 0.4764 - val_accuracy: 0.8380 - val_loss: 0.3712
 Epoch 2/10
 815/815 ————— 3s 4ms/step - accuracy: 0.8508 - loss: 0.3590 - val_accuracy: 0.8533 - val_loss: 0.3551
 Epoch 3/10
 815/815 ————— 4s 5ms/step - accuracy: 0.8614 - loss: 0.3434 - val_accuracy: 0.8567 - val_loss: 0.3502
 Epoch 4/10
 815/815 ————— 5s 6ms/step - accuracy: 0.8659 - loss: 0.3410 - val_accuracy: 0.8607 - val_loss: 0.3442
 Epoch 5/10
 815/815 ————— 4s 5ms/step - accuracy: 0.8622 - loss: 0.3386 - val_accuracy: 0.8613 - val_loss: 0.3410
 Epoch 6/10
 815/815 ————— 3s 4ms/step - accuracy: 0.8679 - loss: 0.3308 - val_accuracy: 0.8602 - val_loss: 0.3377
 Epoch 7/10
 815/815 ————— 4s 5ms/step - accuracy: 0.8662 - loss: 0.3307 - val_accuracy: 0.8645 - val_loss: 0.3327
 Epoch 8/10
 815/815 ————— 5s 6ms/step - accuracy: 0.8732 - loss: 0.3156 - val_accuracy: 0.8667 - val_loss: 0.3307
 Epoch 9/10
 815/815 ————— 4s 4ms/step - accuracy: 0.8709 - loss: 0.3184 - val_accuracy: 0.8679 - val_loss: 0.3270
 Epoch 10/10
 815/815 ————— 4s 4ms/step - accuracy: 0.8690 - loss: 0.3205 - val_accuracy: 0.8670 - val_loss: 0.3242
 204/204 ————— 2s 5ms/step
 204/204 ————— 1s 3ms/step

LSTM Model Performance



```
print('Accuracy:', accuracy)
print('\n'Precision:', precision)
print('\n'Recall:', recall)
print('\n'F1 Score:', f1)
print('\n'AUC:', auc)
```

Accuracy: 0.8669633266840571

Precision: 0.7948979591836735

Recall: 0.5391003460207613

F1 Score: 0.6424742268041237

AUC: 0.8793795640356717